MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión A del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_hernandez_mauricio
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/hernandez_mauricio/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué diferencia hay entre una clase y un objeto? Usa la idea de plano e instancia.

Tu respuesta

```
La clase es la plano abstracto donde existe el objeto, mientras que el objeto es la instancia
que se crea abstrayendo un objeto de la vida real a la maquina.
```

Corrección. Usa bien la idea de plano e instancia: la clase es el plano y el objeto es la instancia. Le falta decir que la clase define atributos y métodos y que cada objeto tiene su propio estado, es decir, sus propios valores de los atributos.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué es la firma de un método y qué partes la forman? Explica si incluye el tipo de retorno.

Tu respuesta

```
La firma del metodo es la forma en la que se identifica el metodo, este estrictamente tiene
que estar formado por el nombre y el tipo de dato que retorna el metodo.
```

Corrección. Es correcto que la firma identifica al método y que incluye su nombre. Sin embargo, la firma no incluye el tipo de retorno: la forman el nombre y los tipos de los parámetros, en orden, y la respuesta omite los parámetros.

Pregunta 0.3 · Preguntas abiertas

Pregunta

Compara la anatomía de una variable local y un atributo de instancia: dónde se declaran, valor inicial y cuánto viven.

Tu respuesta

```
El atributo de instancia se puede declarar al inicio de la clase y este durara todo el transcurso de ejecucion del programa, su valor inicial el nulo hasta que se le asigne uno. MIentas que la variable local solo puede ser declarada dentro de un metodo con un valor cualesquiera que se le asigne y que solo vivira desde que el metodo inicia hasta que el metodo haya terminado.
```

Corrección. Ubica bien dónde se declaran (el atributo en la clase, la local dentro del método) y explica bien que la local vive solo mientras se ejecuta el método. Error conceptual: el atributo de instancia vive lo que vive su objeto, no todo el programa (eso describe a un atributo static). Además, el valor por defecto es null solo para referencias (0 o false en primitivos), y falta decir que la local debe recibir un valor antes de leerse.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué significa null en una referencia? Explica qué ocurre al llamar un método sobre esa referencia.

Tu respuesta

```
Null en una variable de referencia, quiere decir que la ubicacion de la referencia no ha sido encontrada.
```

Corrección. La idea de que la referencia no encuentra nada se acerca a lo correcto: null significa que la referencia no apunta a ningún objeto. Falta la segunda parte: llamar un método de instancia sobre esa referencia hace que el programa truene al ejecutarse (NullPointerException), aunque asignar null por sí solo no falla.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Cuándo conviene usar for-each para recorrer un arreglo y cuándo necesitas un recorrido con índice?

Tu respuesta

Sin respuesta.

Corrección. Sin respuesta.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
StringBuilder ruta = new StringBuilder("S");
StringBuilder alias = ruta;
```

```
for (int i = 1; i <= 3; i++) {
    alias.append(i / 2);
    System.out.println(ruta);
}
String texto = "S" + 2 + 1;
System.out.println(texto);
double mitad = (double) (7 / 2);
System.out.println(mitad);
```

Tu respuesta

```
0
1
1
121
3
```

Corrección. Las líneas 1 a 3 solo muestran el dígito que se agrega (0, 1, 1). Como ruta y alias son el mismo StringBuilder, se acumula: S0, S01, S011. "121" debía ser S21, porque "S"+2+1 concatena de izquierda a derecha, y "3" debía ser 3.0, porque el cast a double convierte el resultado 3.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double medida = 6.8;
int unidades = medida;
System.out.println(unidades);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
Si funciona, 6
```

Corrección. Incorrecto: no compila, porque Java no convierte en automático un double a int, ya que se perdería precisión; haría falta un cast explícito (int).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = {2, 5, 8};
int indice = canales.length;
System.out.println(canales[indice]);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
si funciona, 2/5/8
```

Corrección. Incorrecto: compila pero truena. canales.length vale 3 y el último índice válido es 2, así que canales[3] lanza ArrayIndexOutOfBoundsException sin imprimir nada.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = new String("F2");
System.out.println(codigo == "F2");
System.out.println(codigo.equals("F2"));
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
si Funciona, false/true
```

Corrección. La clasificación (funciona) y la salida false/true son correctas. Falta explicar la regla: == compara si son el mismo objeto, y new String crea uno distinto, mientras que equals compara el contenido.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada multímetro conserva su inventario y nace sin responsable. Se cuenta el total de prestados. Escribe prestar(persona) y una consulta del responsable. Supón que solo se llama prestar con un aparato disponible y un nombre válido.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿La cuenta de prestados pertenece a cada aparato o a todos? ¿Qué dato nunca cambia?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
import Scanner;
public class Prestar{
static final InventarioTotal = 5;
static int multimetros = 5;
String nombre;
static void Prestar(String nombre){
this.nombre = new nombre();
}
static void Cambio{
if(multimetros > 0){
System.out.println("Para el prestamo de multimetros, debera dejar su nombre completo");
System.out.Scanner();
} else{
System.out.println("Multimetros agotados");
}
return null;
}
static void Consulta{
System.out.println("Numero de multimetros disponibles" + multimetros);
}
```

Justificaciones

Sin respuesta.

> **Corrección.** No hay justificaciones. Atributos: solo hay un atributo de instancia (nombre), multimetros cuenta los disponibles y no los prestados, y la constante no tiene tipo ni representa el inventario de cada aparato. El constructor está escrito como método static void y usa new nombre(); Cambio usa if y Scanner y no asigna responsable ni suma préstamos, Consulta muestra el estado de forma simple, y el reto no se responde. Lo esperado: inventario final de instancia, responsable de instancia mutable, static int prestados compartido, y prestar(persona) que asigne y sume uno.

R4 · Código con repetición

Pregunta

Un equipo guarda sus folios como enteros no negativos. Devuelve cuántos dígitos tiene folio usando while y división entre 10. El folio 0 tiene un dígito. No conviertas el número a texto.

```
static int contarDigitos(int folio)
```

Ejemplos (argumentos → retorno):

4072 → 4 · 0 → 1 · 8 → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
No me acuerdo D:
```

> **Corrección.** Sin respuesta: el alumno escribió que no lo recordaba. Lo esperado era tratar el 0 como un dígito y, mientras folio > 0, hacer folio /= 10 y contar cada división, para luego devolver la cuenta.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la carga de un equipo. Usa switch clásico sobre carga / 10. Puedes validar antes con if. Devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String nivelCarga(int carga)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 50	"INVALIDA"
0 a 19	"BAJA"
20 a 39	"MEDIA"
40 a 50	"ALTA"

Tu respuesta

```
public class Carga{
int carga;
if (carga/10 > 0 && carga/10 < 50){
System.out.println("¿Cual es la carga de tu equipo?");
System.out.Scanner();
switch (carga/10){
case ´A´:
System.out.println("ALTA");
break;
case ´B´:
System.out.println("MEDIA");
break;
case ´C´:
System.out.println("BAJA");
```

```
break;
default "INVALIDA";
}} else{
System.out.println("INVALIDA");
}
return carga;
}
}
```

Corrección. Falta el método static String nivelCarga(int carga): el código va suelto dentro de una clase, imprime y lee del teclado, y devuelve un int. Acierta al validar con if antes de un switch clásico sobre carga/10, pero valida carga/10 en lugar de carga (de 0 a 50 con ambos extremos) y usa los casos 'A', 'B' y 'C', que no son cocientes. Lo correcto era: case 0 y 1 devuelven "BAJA", 2 y 3 "MEDIA", 4 y 5 "ALTA".

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.